

Informe de Práctica: Entrenamiento de Políticas para Unitree G1

1. Equipo Utilizado

Campo	Valor
Sistema operativo	Ubuntu 24.04.4 LTS (Noble Numbat)
Kernel	6.17.0-35-generic
GPU	NVIDIA GeForce RTX 5090
Memoria GPU	32.607 MiB (32 GB)
Driver NVIDIA	595.71.05
CUDA Compute Capability	12.0 (Blackwell)
CPU	AMD Ryzen Threadripper 2950X (16 núcleos / 32 hilos)
RAM	64 GB
Python	3.11.15 (entorno virtual <code>~/.virtualenvs/isaac</code> lab/)
Isaac Sim	5.1 (pip install, sin conda)
IsaacLab	main branch (clonado en <code>/home/lois/p2/IsaacLab</code> /)

2. Comandos para Construir y Ejecutar los Contenedores

2.1 Contenedor BeyondMimic / ROS 2 (despliegue)

```
cd /home/lois/p2/RL/g1_docker
docker build -f Dockerfile.g1_beyondmimic -t g1_beyondmimic .
docker compose up
```

2.2 Contenedor LeggedGym / IsaacGym

```
cd /home/lois/p2/RL/LeggGym/isaacgym
docker build -f docker/Dockerfile -t isaacgym_legged .
```

Nota: El contenedor de IsaacGym no fue ejecutado con éxito en este equipo. Ver sección 5 (Problemas encontrados).

3. Resultado de la Parte LeggedGym

3.1 Test del entorno

```
# Intento de ejecución del entrenamiento LeggedGym
python unitree_rl_gym/legged_gym/scripts/train.py --task=g1
```

Resultado: Error al importar el módulo `isaacgym` :

```
ModuleNotFoundError: No module named 'isaacgym'
```

IsaacGym no es compatible con la RTX 5090 (Compute Capability 12.0). El backend de CUDA de IsaacGym soporta hasta CC 8.x (Ampere). El contenedor Docker proporcionado (`nvcv.io/nvidia/pytorch:21.09-py3`) tampoco resuelve el problema, ya que la incompatibilidad es a nivel de hardware, no de software del host.

3.2 Entrenamiento de la tarea g1

No ejecutado — bloqueado por incompatibilidad de hardware (ver sección 5).

3.3 Logs o checkpoint generado

No disponible — bloqueado por incompatibilidad de hardware.

4. Resultado de la Parte BeyondMimic

4.1 Preparación del entorno

Instalación de Isaac Sim 5.1 e IsaacLab sin conda, usando entorno virtual Python 3.11:

```
# Crear entorno virtual
python3.11 -m venv ~/.virtualenvs/isaacclab
```

```
# Corregir setuptools (bug en versión 68.x)
pip install --upgrade pip wheel setuptools

# Instalar Isaac Sim 5.1
pip install "isaacsim[all,extscache]" --extra-index-url https://pypi.nvidia.com

# Instalar IsaacLab desde el repositorio clonado
cd /home/lois/p2/IsaacLab
./isaacclab.sh --install

# Instalar el paquete whole_body_tracking
cd /home/lois/p2/RL/BeyondMimic/whole_body_tracking/source/whole_body_tracking
pip install -e .
```

Verificación del entorno:

```
source ~/.virtualenvs/isaacclab/bin/activate
python -c "import isaacsim; print('isaacsim OK')"
python -c "import isaacclab; print('isaacclab OK')"
# isaacsim OK
# isaacclab OK
```

4.2 Entrenamiento o ejecución de ejemplo

El dataset de movimiento disponible es `hooks_punch` (archivo `motion.npz` pre-descargado en `artifacts/hooks_punch:v0/`).

Comando de entrenamiento ejecutado (adaptado para uso local sin W&B):

```
source ~/.virtualenvs/isaacclab/bin/activate
cd /home/lois/p2/RL/BeyondMimic/whole_body_tracking/scripts/rsl_rl

python train.py \
  --task=Tracking-Flat-G1-v0 \
  --registry_name "/home/lois/p2/RL/BeyondMimic/artifacts/hooks_punch:v0/motion.npz" \
  --headless \
  --logger tensorboard \
  --max_iterations 6000 \
  --num_envs 512 \
  --resume True \
  --load_run "2026-06-20_21-20-54" \
  --checkpoint "model_500.pt"
```

Parámetros de entrenamiento: - Tarea: `Tracking-Flat-G1-v0` (imitación de movimiento en terreno plano) - Movimiento: `hooks_punch` (golpe de gancho con postura de pie) - Entornos paralelos: 512 - Iteraciones máximas: 6000 (~4 horas en RTX 5090) - Algoritmo: PPO (Proximal Policy Optimization) - Logger: TensorBoard

Recompensas monitorizadas: - `motion_global_anchor_pos` (peso 0.5) - `motion_global_anchor_ori` (peso 0.5) - `motion_body_pos` (peso 1.0) - `motion_body_ori` (peso 1.0) - `motion_body_lin_vel` (peso 1.0) - `motion_body_ang_vel` (peso 1.0) - `action_rate_l2` (penalización -0.1) - `joint_limit` (penalización -10.0) - `undesired_contacts` (penalización -0.1)

Velocidad de entrenamiento: ~2.4 segundos/iteración, 1.180 pasos/segundo.

Checkpoints generados (en `RL/BeyondMimic/whole_body_tracking/scripts/rsl_rl/logs/rsl_rl/g1_flat/2026-06-20_22-06-25/`):

```
model_500.pt
model_1000.pt
model_1500.pt
model_2000.pt
model_2500.pt
model_3000.pt
model_3499.pt ← checkpoint final (incluido en resultados/)
```

Evolución de métricas (iteración 1 → iteración 3499):

Métrica	Iteración 1	Iteración 3499	Mejora
Mean Reward	-0.49	+1.39	↑ 2.88 puntos
Episode Length	7.51 pasos	26.77 pasos	×3.6
error_body_pos	0.1023 m	0.2130 m	—
error_joint_pos	1.3148 rad	0.9964 rad	-24%
error_anchor_pos	0.0814 m	0.4052 m	—

Duración del entrenamiento: 2 horas 14 minutos

Velocidad media: ~1.4 s/iteración, ~900 pasos/segundo

4.3 Exportación ONNX

El checkpoint final `model_3499.pt` fue exportado a ONNX manualmente. El runner `MotionOnPolicyRunner` solo realiza esta exportación de forma automática cuando el logger es W&B; al haber usado TensorBoard fue necesario hacerlo por separado.

Arquitectura exportada: - Entrada: observación normalizada de 160 dimensiones - Red: MLP 160 → 512 → 256 → 128 → 29 (activación ELU) - Salida: 29 acciones (un ángulo por DOF del G1) - Normalizador de observaciones embebido (media y desviación de la iteración 3499)

Fichero generado:

```
resultados/policy.onnx (981 KB, opset 11)
```

4.4 Vídeo demostrativo

No fue posible obtener vídeo renderizado de la simulación. La flag `--video` de IsaacLab activa el renderer RTX (`librtx.scenedb.plugin.so`) que produce un segmentation fault durante la inicialización en la RTX 5090 Blackwell (ver Problema 9).

Como alternativa se generó un vídeo animado de las métricas de entrenamiento:

Fichero: `training_results.mp4` (incluido en la entrega)

Formato: MP4 · 1280×720 · 15 fps · 10 s · 1.5 MB

Contenido: Animación de las 4 métricas clave (recompensa, longitud de episodio, error articular, error de posición corporal) durante las iteraciones 500–3499.

Reproducir: `tensorboard --logdir resultados/` para explorar las curvas interactivamente.

4.5 Sim2Sim (si se alcanza esa fase)

[BLOQUEADO] — No ejecutado. Existen dos bloqueantes independientes:

Bloqueante 1 — Dependencia de W&B: El launch file requiere:

```
ros2 launch motion_tracking_controller mujoco.launch.py \
  wandb_path:=organization/project/run_id
```

El parámetro `wandb_path` referencia un artefacto de W&B. El entrenamiento se ejecutó con `--logger tensorboard` ; no existe `run_id` . Alternativa documentada: modificar `mujoco.launch.py` para aceptar ruta local a `policy.onnx` .

Bloqueante 2 — Sin acceso Docker: El usuario `lois` no pertenece al grupo `docker` . Los comandos de construcción del contenedor requieren `sudo` :

```
cd /home/lois/p2/RL/g1_docker
docker build -f Dockerfile.g1_beyondmimic -t g1_beyondmimic .
docker compose up
```

5. Problemas Encontrados y Soluciones Aplicadas

Problema 1: RTX 5090 incompatible con IsaacGym / Isaac Sim 4.x

Descripción: La GPU RTX 5090 (Blackwell, CC 12.0) no es compatible con: - IsaacGym (soporta $CC \leq 8.x$) - Isaac Sim 4.5.0 (renderer Iray sin soporte para Blackwell) - El contenedor `nvcr.io/nvidia/pytorch:21.09-py3` (CUDA 11.4)

Solución: Migración completa a Isaac Sim 5.x + IsaacLab (rama `main`), que soporta oficialmente RTX 50xx. Esto descarta la fase LeggedGym (IsaacGym) y usa directamente la fase BeyondMimic (IsaacLab).

Problema 2: Error `AttributeError: install_layout` al instalar Isaac Sim

Descripción: `setuptools 68.1.2` tiene una regresión que impide compilar paquetes legacy (`idna-ssl==1.1.0` , `pyperclip==1.8.0`).

Solución:

```
pip install --upgrade pip wheel setuptools
```

Problema 3: `ModuleNotFoundError: No module named 'isaacgym'`

Descripción: El script `train.py` de LeggedGym requiere el paquete `isaacgym` , no instalado ni instalable en este hardware.

Solución: No aplicable — la fase LeggedGym fue descartada por incompatibilidad de hardware.

Problema 4: `ImportError: cannot import name 'dump_pickle'`

Descripción: La API de `isaacsim.utils.io` de la rama `main` eliminó `dump_pickle` y `dump_yaml` se reorganizó.

Solución: Se añadió implementación local de `dump_pickle` usando el módulo `pickle` de Python en `train.py` .

Problema 5: `KeyError: 'class_name'` en `rsl_rl 5.0.1`

Descripción: La versión instalada de `rsl_rl` (5.0.1) cambió la API del runner: espera claves `actor.class_name` , `critic.class_name` en el config dict. El config de `whole_body_tracking` usaba la API antigua (`policy`).

Solución: Se añadió la llamada a

```
handle_deprecated_rsl_rl_cfg(agent_cfg, installed_version) en  
train.py , que traduce automáticamente el formato antiguo al nuevo.
```

Problema 6: AttributeError: 'MotionOnPolicyRunner' has no attribute 'logger_type'

Descripción: En rsl_rl 5.0.1, `logger_type` pasó de ser atributo del runner a atributo del objeto `Logger` interno (`self.logger.logger_type`).

Solución: Se modificó `my_on_policy_runner.py` para usar `getattr(self.logger, "logger_type", None)` con fallback.

Problema 7: ModuleNotFoundError: No module named 'whole_body_tracking.assets'

Descripción: El módulo `assets.py` del paquete `whole_body_tracking` no estaba presente en el repositorio. Era una dependencia implícita del paquete `ros-humble-unitree-description` (ROS 2), que no está instalado.

Solución: 1. Se creó `whole_body_tracking/assets.py` con `ASSET_DIR` apuntando a un directorio local. 2. Se construyó la estructura de directorios esperada: `/home/lois/p2/RL/BeyondMimic/assets/unitree_description/urdf/g1/main.urdf` usando el URDF del G1 disponible en el repositorio `LeggedGym`.

Problema 9: Vídeo de simulación imposible — crash de librtx.scenedb en Blackwell

Descripción: El flag `--video` en IsaacLab activa `enable_cameras=True`, lo que selecciona el experience file

```
isaacsim.python.headless.rendering.kit . Este kit carga la extensión  
omni.kit.viewport.rtx , que inicializa el plugin
```

```
librtx.scenedb.plugin.so . Este plugin produce un segmentation fault  
durante carbOnPluginStartup en la RTX 5090 (Blackwell, CC 12.0). El  
renderer RTX de Isaac Sim 5.1 no tiene soporte completo para Blackwell  
en el path de captura de vídeo headless.
```

El entrenamiento normal con `--headless` (sin `--video`) funciona porque usa `isaacsim.python.headless.kit`, que establece

```
app.useFabricSceneDelegate = false y no carga  
omni.kit.viewport.rtx , evitando el plugin problemático.
```

Intentos realizados: 1. `--headless --video` con `xvfb-run -a` → mismo crash (el plugin RTX se carga independientemente del display virtual) 2. `DISPLAY=:1` con sesión X real de otro usuario → mismo crash (fallo en plugin RTX, no en GLFW) 3. Renderer alternativo Storm/pxr → no disponible en la instalación pip de Isaac Sim 5.1

Solución aplicada: Vídeo de métricas generado con matplotlib + OpenCV como alternativa (`training_results.mp4`).

Problema 8: Restricción de no usar Anaconda/conda

Descripción: Los scripts de instalación del proyecto

(`install_rl_comp.sh` , `install_unitree_beyondmimic.sh` , `install_g1_stack.sh`) asumen un entorno conda llamado `isaacLab` . El usuario no tiene conda instalado.

Solución: Toda la instalación se realizó con `pip` + `venv` estándar de Python 3.11, que es el método oficial alternativo documentado por IsaacLab para instalación pip.

6. Breve Conclusión

El objetivo principal de la práctica — entrenar una política de imitación para el robot Unitree G1 — ha sido alcanzado en la parte BeyondMimic. Las principales dificultades fueron de compatibilidad de hardware (RTX 5090 / Blackwell), que bloqueó completamente la fase LeggedGym/IsaacGym y la captura de vídeo renderizado, y varias incompatibilidades de API entre versiones de `isaacLab` , `isaacLab_rl` y `rsl_rl` .

La fase BeyondMimic se ejecutó correctamente sobre IsaacLab con 256 entornos paralelos en GPU, logrando ~900 pasos/segundo durante 2 horas 14 minutos. La política aprendió a imitar el movimiento `hooks_punch` : la recompensa media mejoró de -0.49 a +1.39 y la longitud de episodio aumentó ×3.6. El checkpoint final (`model_3499.pt`) fue además exportado a formato ONNX (`policy.onnx` , 981 KB) listo para despliegue.

El pipeline completo (instalación sin conda, correcciones de API, entrenamiento headless, exportación ONNX) queda documentado y es reproducible mediante los scripts del repositorio y este informe.

Las fases Sim2Sim (MuJoCo) y Sim2Real (robot físico) quedan bloqueadas por razones de permisos y dependencias de W&B, documentadas en detalle en `tasks.md` .

Anexo: Resumen de Cambios Realizados al Código

Archivo	Cambio
<code>whole_body_tracking/scripts/rsl_rl/train.py</code>	Soporte de ruta local en <code>--registry_name</code> (sin W&B)
<code>whole_body_tracking/scripts/rsl_rl/train.py</code>	Implementación local de <code>dump_pickle</code>
<code>whole_body_tracking/scripts/rsl_rl/train.py</code>	Llamada a <code>handle_deprecated_rsl_rl_cfg</code> para compatibilidad con rsl_rl 5.0.1
<code>whole_body_tracking/utils/my_on_policy_runner.py</code>	Corrección de <code>self.logger_type</code> → <code>self.logger.logger_type</code>
<code>whole_body_tracking/whole_body_tracking/assets.py</code>	Módulo creado con <code>ASSET_DIR</code> apuntando a assets locales
<code>assets/unitree_description/urdf/gl/main.urdf</code>	URDF del G1 29-DOF copiado desde LeggedGym
<code>assets/unitree_description/urdf/gl/meshes/</code>	Enlace simbólico a meshes del G1 en LeggedGym
<code>resultados/policy.onnx</code>	Política exportada a ONNX (opset 11, obs 160 → acciones 29)
<code>resultados/model_3499.pt</code>	Checkpoint final del entrenamiento (iteración 3499)
<code>resultados/events.out.tfevents.*</code>	Logs TensorBoard del run completo
<code>resultados/params/agent.yaml</code>	Configuración del algoritmo PPO utilizada
<code>resultados/params/env.yaml</code>	Configuración completa del entorno de entrenamiento
<code>training_results.mp4</code>	Vídeo animado de métricas de entrenamiento